You can also use ERB to construct HTML fragments on the server that can then be used by the browser to perform partial page updates. This is great for creating dynamic Hotwire interfaces. We talk about these starting in *Iteration F2: Creating a Hotwired Cart*.

Rails also provides libraries to construct XML or JSON documents using Ruby code. The structure of the generated XML or JSON automatically follows the structure of the code.

And the Controller!

The Rails controller is the logical center of your application. It coordinates the interaction among the user, the views, and the model. However, Rails handles most of this interaction behind the scenes; the code you write concentrates on application-level functionality. This makes Rails controller code remarkably easy to develop and maintain.

The controller is also home to a number of important ancillary services:

- It's responsible for routing external requests to internal actions. It handles people-friendly URLs extremely well.
- It manages caching, which can give applications orders-of-magnitude performance boosts.
- It manages helper modules, which extend the capabilities of the view templates without bulking up their code.
- It manages sessions, giving users the impression of ongoing interaction with our applications.

We've already seen and modified a controller in *Hello, Rails!*, and we'll be seeing and modifying a number of controllers in the development of a sample application, starting with the products controller in *Iteration C1: Creating the Catalog Listing*.